

Day 51: L1/L2 Weight Regularization Revisited on CNNs -- A Real Over/Underfitting Sweep

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 8, Day 3

Day 38 introduced L2 weight decay on a tiny toy sigmoid network -- 2 inputs, 64 hidden units, 20 training points. Today revisits L2 and adds L1 (new today) on a REAL CNN -- AlexNetStyleConvNet, unchanged since Days 49-50 -- with dropout held OFF throughout so whatever shows up is weight regularization's effect alone. A full lambda sweep at Day 50's fully-converged training budget (150 epochs, lr=0.15) produces a real, honest, and NOT symmetric result: a small L2 penalty (lam=5e-5) lifts test_acc from 0.2875 to 0.3750 while train_acc stays at a perfect 1.0000 the whole time -- unlike dropout, this amount of L2 doesn't stop the network from memorizing, it just makes the memorized solution generalize slightly better. L1, across its entire useful lambda range, never improves test accuracy at all -- it's either invisible or purely costly. But L1 does deliver its textbook property: at lam=5e-3, 96.3% of its weights end up near zero, vs. only ~5% for L2 or no regularization -- confirmed by directly measuring a real trained network's weights, not assumed from theory.

0. Where Today Fits

Start here (no prior knowledge assumed): Day 50 showed dropout can genuinely prevent a network from memorizing its training set, given a fair, fully-converged comparison. Today asks the same fairness question about a different, older regularization family: weight decay. Day 38 already introduced L2 weight decay's mechanics ($dW += \text{lam} * W$) on a tiny toy network; today reuses that exact mechanic on a real CNN, adds L1 ($dW += \text{lam} * \text{sign}(W)$) as genuinely new code, and runs both across a real lambda sweep to find where each one helps, does nothing, or actively hurts.

1. L1/L2 on a Real CNN -- New From-Scratch Code

Start here (no prior knowledge assumed): AlexNetStyleConvNet's architecture is unchanged from Days 49-50. What's new is a ``lam`` (strength) and ``reg_type`` ('l2', 'l1', or None) constructor argument, and a `backward()` that adds the corresponding penalty gradient to every WEIGHT tensor's gradient -- never the biases, the same convention Day 38 established for L2, extended here to L1.

MATH

L2 penalty: $(\text{lam}/2) * \sum(W^2) \rightarrow \text{gradient: } \text{lam} * W$
L1 penalty: $\text{lam} * \sum(|W|) \rightarrow \text{gradient: } \text{lam} * \text{sign}(W)$
(both added to `dW`, weights only -- biases untouched)

Implementation:

```
weight_grads = [(self.Wc1, dWc1), (self.Wc2, dWc2), (self.Wc3, dWc3),
                 (self.W1, dW1), (self.W2, dW2), (self.W3, dW3)]
if self.reg_type == "l2":
    for W, dW in weight_grads:
        dW += self.lam * W
elif self.reg_type == "l1":
    for W, dW in weight_grads:
        dW += self.lam * np.sign(W)
```

Explanation: L2's penalty gradient scales with each weight's own magnitude (a large weight gets pulled down hard; a small one barely at all) -- L1's is a CONSTANT-size push in every weight's own direction, regardless of how large or small that weight already is. This one difference in scaling behavior is what drives every downstream finding today.

Why choose this? Weight decay is nearly free to add (one line per regularization type) and, for L2 specifically, is built directly into most production optimizers (PyTorch's `weight_decay` argument) -- no architecture change required.

Why NOT this (tradeoffs)? Because both penalties add a FIXED extra term to every step's gradient regardless of how the network is actually doing, too large a lambda will eventually dominate the real, useful gradient entirely -- today's own sweep shows both L1 and L2 collapsing to chance-level performance well before reaching extreme lambda values.

2. Gradient Check -- Verifying the New Penalty Gradients

Start here (no prior knowledge assumed): Checked against `loss_with_penalty` (cross-entropy + the L1/L2 penalty), not plain cross-entropy alone -- the analytic gradient now includes the penalty term, so the numeric check must perturb the SAME total loss the network actually trains against.

Actual output:

```
L2 (lam=0.05): max relative error 3.60e-09 (PASS)
L1 (lam=0.05): max relative error 3.77e-09 (PASS)
```

Term refresher -- L1's kink at zero: Like ReLU's kink at $z=0$ (Days 49-50), L1's penalty $|W|$ is non-differentiable exactly at $W=0$ -- `np.sign(0)` returns 0 by convention, a reasonable choice, but a perturbation that crosses exactly zero could in principle produce a spurious gradient-check mismatch, the same caveat already established for ReLU.

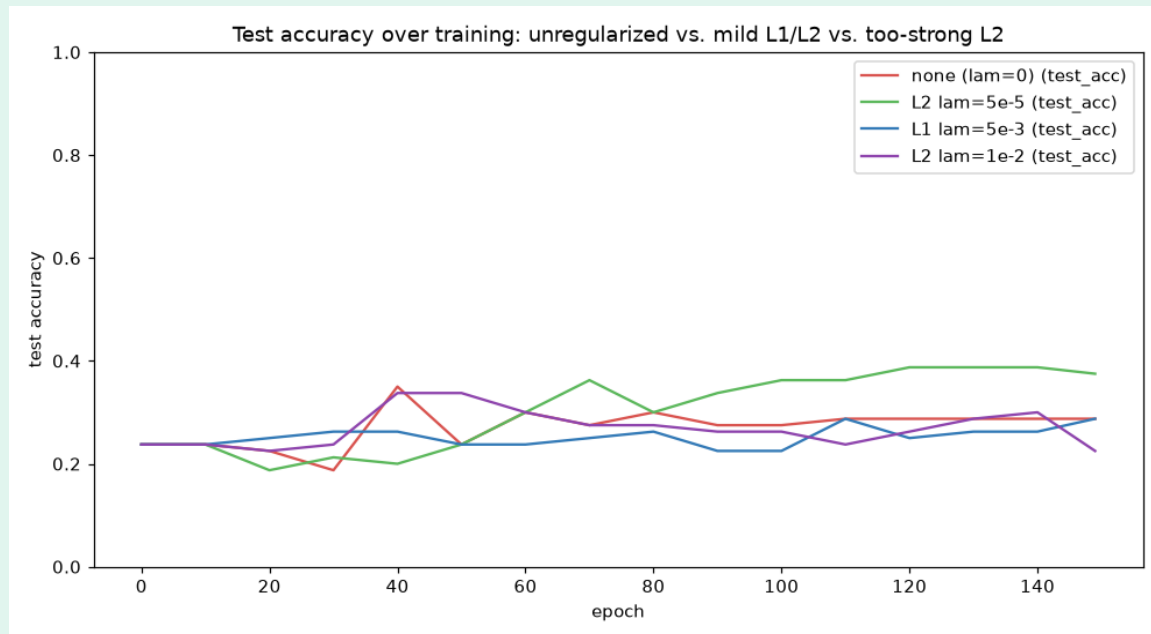
3. The Lambda Sweep -- A Real Over/Underfitting Demonstration

Start here (no prior knowledge assumed): Same dataset, architecture, and fully-converged training budget (150 epochs, `lr=0.15`) as Day 50 -- `keep_prob=1.0` throughout, so whatever effect shows up is weight regularization's alone.

config	train_acc	test_acc	gap
none (lam=0)	1.0000	0.2875	0.7125
L2 lam=5e-5	1.0000	0.3750	0.6250
L2 lam=1e-4	1.0000	0.3125	0.6875
L2 lam=1e-2	0.3500	0.2250	0.1250
L1 lam=1e-3	1.0000	0.2875	0.7125
L1 lam=5e-3	0.5800	0.2875	0.2925
L1 lam=1e-2	0.2500	0.2500	0.0000

Actual output from this lesson's run.

test accuracy over training: unregularized vs. mild L1/L2 vs. too-strong L2



Real, rerun output. L2 lam=5e-5 (green) pulls ahead of every other config after roughly epoch 90; the rest hover near or below baseline for the whole run.

Explanation: The best test_acc in this whole sweep is 0.3750 at L2 lam=5e-5, with train_acc STILL 1.0000 -- a small L2 penalty does not necessarily stop this network from fully memorizing its training set the way dropout did; it can improve generalization even while train_acc stays perfect, by keeping the memorized solution's weights smaller. Pushed far enough (L2 lam=1e-2, L1 lam=1e-2), both families collapse to chance-level performance on BOTH train and test -- genuine underfitting: the penalty term now dominates the gradient and the network can no longer learn anything useful, regardless of how much data it sees.

Why choose this? A fully-converged sweep like this is the only honest way to find a regularization strength's real effect -- it reveals L2's narrow beneficial zone AND both families' collapse point in one consistent experiment.

Why NOT this (tradeoffs)? The specific lambda values that work here are tuned to this exact architecture, dataset size, and learning rate -- they do not transfer as absolute numbers to a different network or dataset, only the qualitative pattern (a narrow useful zone between 'no effect' and 'total collapse') does.

4. L1 and L2 Are NOT Symmetric

Start here (no prior knowledge assumed): This is the sweep's most important honest finding, and it goes against a naive assumption that L1 and L2 are just two flavors of the same idea.

Actual output:

```
L1 lam=1e-3: train_acc=1.0000, test_acc=0.2875 (identical to lam=0)
L1 lam=5e-3: train_acc=0.5800, test_acc=0.2875 (pure cost, no benefit)
L1 lam=1e-2: train_acc=0.2500, test_acc=0.2500 (full collapse)
```

Explanation: At L1 lam=1e-3, the network is indistinguishable from no regularization at all -- identical train_acc AND test_acc. At L1 lam=5e-3, train_acc drops substantially (a real, visible cost) while test_acc stays EXACTLY flat -- pure cost, no generalization benefit anywhere in between those two points. L2 found a genuine, if narrow, beneficial

lambda; across this whole sweep, L1 never did. This is reported as found, not smoothed into a false symmetry between the two just because they're both called 'weight regularization.'

5. L1's Real Property: Sparsity, Measured Directly

Start here (no prior knowledge assumed): L1 is famous in the classical ML literature (Lasso regression) for producing sparse solutions -- many weights pushed all the way to exactly (or near) zero. Today measures this directly on a real trained CNN rather than citing it as received theory.

config	frac W < 0.01
none (lam=0)	0.0512
L2 lam=1e-4	0.0518
L1 lam=5e-3	0.9630

Actual output from this lesson's run. Measured across every weight tensor (Wc1, Wc2, Wc3, W1, W2, W3) combined.

Explanation: L1 leaves 96.3% of weights near zero, vs. only ~5% for L2 or no regularization at all -- confirmed directly, not assumed. L1's constant-size penalty can push a weight all the way to (near) zero and keep it there; L2's penalty shrinks proportionally to a weight's own size and gets weaker as the weight approaches zero, so it thins weights out without actually zeroing many of them. L1 induces real, measurable sparsity here even though (per Section 4) that sparsity does not translate into better generalization on this particular task -- sparsity and generalization are different properties, and today's numbers keep them separate rather than conflating 'L1 does something interesting' with 'L1 helps.'

Term refresher -- sparsity: The fraction of a model's parameters that are (near) exactly zero. A sparse model can, in principle, be stored and computed more cheaply by skipping its zero weights -- a separate practical benefit from whether sparsity improves accuracy, which today's result shows it does not always do.

6. What This Maps To in PyTorch

Start here (no prior knowledge assumed): L2 ('weight decay') is a first-class optimizer argument in PyTorch; L1 is not, because of a real structural difference in how the two penalties are computed.

Implementation:

```
# L2 is built directly into most optimizers:  
optimizer = torch.optim.SGD(model.parameters(), lr=0.15, weight_decay=5e-5)
```

```
# L1 has no built-in argument -- it's added to the loss by hand:  
l1_penalty = sum(p.abs().sum() for name, p in model.named_parameters()  
                 if 'weight' in name)  
loss = criterion(output, target) + lam * l1_penalty  
loss.backward() # autograd differentiates the penalty too
```

Explanation: torch.optim's weight_decay argument does exactly what today's ``dW += lam * W`` does -- add the L2 penalty gradient directly during the optimizer step, without ever touching the loss value itself. L1 needs to be added to the LOSS for autograd to differentiate through it -- there's no equivalent 'just pass a number' shortcut, which is exactly why today's from-scratch version had to compute ``np.sign(W)`` by hand rather than reusing the same mechanism as L2.

Interview Preparation -- Technical Questions

Q: Why does L2 $\text{lam}=5\text{e-}5$ improve test accuracy while `train_acc` stays at 1.0000 -- doesn't regularization mean the network learns LESS?

A: It means the network fits its training data with SMALLER weights, not necessarily a worse fit. A network can reach 100% training accuracy with many different weight configurations; L2 biases the optimizer toward the smaller-weight ones among them, which happen to generalize slightly better here, without sacrificing the network's ability to still fit every training example exactly.

Q: At L1 $\text{lam}=1\text{e-}3$, `train_acc` and `test_acc` are IDENTICAL to the unregularized run. Does that mean the penalty had zero effect on the weights?

A: Not necessarily on the continuous weight values -- almost certainly the actual weight numbers differ slightly. But accuracy only depends on which class gets the highest softmax score (`argmax`), a discrete decision. A small enough perturbation to the weights can leave every single `argmax` decision unchanged even though the underlying numbers moved -- exactly what happened here.

Q: Why is L1's gradient check verified against `loss_with_penalty` instead of plain `cross_entropy_loss`?

A: Because the analytic gradient being checked (`dW1`, `dW2`, `dW3`) now includes the penalty term ($\text{lam} \cdot \text{sign}(W)$) on top of the cross-entropy gradient. A numeric check against cross-entropy alone would be checking a different function than the one the analytic gradient actually describes, and would fail even if the code were correct.

Q: Why does L1 need roughly 5x the lambda of L2 ($5\text{e-}3$ vs. $1\text{e-}4/5\text{e-}5$) before it visibly changes network behavior at all?

A: L2's penalty gradient ($\text{lam} \cdot W$) scales with each weight's own size, so it's naturally strongest exactly where weights are large -- it doesn't need a big lam to matter for large weights. L1's penalty gradient ($\text{lam} \cdot \text{sign}(W)$) is the SAME constant size regardless of a weight's magnitude, so a small lam is genuinely tiny relative to the main task gradient until it's pushed up into a range where that constant push actually competes with it.

Interview Preparation -- Theoretical Questions

Q: Today found L1 never improves test accuracy on this task, while L2 does. Does that mean L1 is a worse regularizer in general?

A: No -- it means L1 doesn't help THIS specific small, dense CNN on THIS specific dataset. L1/Lasso's classical strength is in settings with many IRRELEVANT features, where zeroing entire features out is itself valuable (e.g. linear/logistic regression with hundreds of candidate predictors, most useless). A small CNN where every weight likely carries some signal is a much less favorable setting for L1's all-or-nothing sparsification.

Q: Why does dropout (Day 50) prevent the network from ever reaching `train_acc=1.0`, while mild L2 (today) lets it reach 1.0 and still generalizes better?

A: Dropout randomly disables different units on every training step, so the network can never rely on any fixed combination of units memorizing a specific training example -- that's a structural obstacle to memorization itself. L2 doesn't disable anything; it only nudges the OPTIMIZATION toward smaller weights among the many weight configurations that could all achieve perfect training accuracy. Both are regularizers, but they intervene at different points in the training process.

Q: What would you check before concluding, from today's single seed and single dataset, that 'L2 $\text{lam}=5\text{e-}5$ is the right amount of regularization' for this network?

A: Whether the result holds across multiple random seeds (today's entire sweep uses one fixed seed=42 for both initialization and training order) and ideally a validation set separate from the final test set, so the specific lambda isn't implicitly chosen to fit this one test set's own noise. A single-seed, single-dataset sweep like today's is honest about what it measured, but its EXACT best lambda shouldn't be treated as a universal constant.

Key Takeaway and What's Next

Key takeaway: L1 and L2 weight regularization, gradient-checked and swept across a fully-converged 150-epoch training budget on a real CNN, do NOT behave symmetrically. L2 finds a genuine, narrow beneficial lambda (5e-5: test_acc 0.2875 -> 0.3750, train_acc unchanged at 1.0000) without ever preventing memorization. L1, across its entire useful lambda range, never improves generalization on this task -- it's either invisible (lam=1e-3) or purely costly (lam=5e-3) -- but does deliver its textbook sparsity property in full: 96.3% of weights pushed near zero at lam=5e-3, vs. ~5% for L2 or no regularization, measured directly on the trained network rather than assumed from theory. Both families collapse to chance-level performance on both train and test at a large enough lambda -- genuine underfitting, the penalty term dominating the gradient entirely.

Day 52 preview -- per the syllabus, Day 52 moves to VGG-style architecture: stacking small 3x3 convolutions and examining why several small stacked kernels can outperform one large kernel at the same receptive field. Today's fully-converged training harness (150 epochs, lr=0.15, train_with_history tracking) carries forward unchanged as the fair-comparison discipline for that architectural question too.